

Rho Combinators as Sparse Linear Algebra

A classical route from reflective concurrency to data-parallel execution

L. G. Meredith

CEO, F1R3FLY.io, and Chief Science Officer, F1R3FLY Industries

23 September 2026 — version 1.2

Abstract

The question this note answers is whether the concurrency of the rho calculus can be mapped onto the concurrency of a graphics processor by way of the name-free combinators, and specifically whether the reduction graph of a combinator term admits a matrix representation that factors into a representation of the rewrite rules and repeated application to a vector encoding of a term. The scope here is strictly classical: no weights, no amplitudes, no quantum reading.

The answer has three parts. First, the factorisation exists and is standard, once the right state space is identified. The companion logic note proves that the multiset of top-level atoms is a complete invariant for structural congruence, so a combinator state *is* an occupation vector and the one-step operator is a sum of ladder-operator monomials, one per rule of a table generated from the shape table, each of degree one more than the largest constructor arity the program uses. The equations of the calculus are quotiented by construction rather than by normalisation.

Second, the three invariants used in the draft’s expressiveness lattice — name closure, atom count, message count — turn out to be exactly the three finiteness conditions the matrix representation requires, and each is controlled by a different axis of the lattice. Name closure controls the index set, so the constructors decide whether the basis is finite or generated on demand. A weighted atom count controls nilpotency, so the destructor decides whether the transfer operator is strictly block upper triangular. The count of messages and stores together controls occupation bounds. The lattice therefore already computes which matrix shape a term admits, and the eager and lazy parts of the operator are separated by the same analysis: seven rules are constant-coefficient and fully schematic, every member of the constructor family requires an encode, and exactly one, the destructor, requires a decode.

Third, and this is the practical finding, the object worth multiplying is not the transition matrix of the reduction graph. It is the incidence matrix of atoms against names, which is term-sized and sparse. Redex finding for every rule is a join on that matrix, conflict detection between redexes is a second sparse product, and a machine step is a maximal independent set in the resulting conflict graph. Firing an independent set is sound with respect to the interleaving semantics, so the number of machine steps is the causal depth of the computation rather than its interleaving length. That ratio is the available speedup, and it is measurable. A reference implementation accompanies the note; its matrix formulation of redex finding agrees with naive enumeration on 400 random states, and on four families of terms the measured ratio of sequential steps to parallel steps is exactly the width of the term.

A phased development plan and seven numbered obligations close the note.

Version 1.1. Two clauses of Theorem 6.1 were false as stated in version 1, and are corrected with no change of numbering. Clause (ii) graded by the plain count of non-message atoms, which the rules for $\mathbf{b}_l$, $\mathbf{b}_r$ and $\mathbf{s}$ preserve; it now grades by the weighted count of the revised [1, Lem. 9.2], and the nilpotency index and Corollary 6.2 change accordingly. Clause (iii) claimed the message count non-increasing without $\mathbf{d}$ and $\mathbf{e}$, which the quote

rule violates; it now bounds messages and stores together. The reference implementation is also corrected to count a symmetric match once, as Proposition 5.3 requires; no measured number changes.

Version 1.2. Draft 3 now translates in two phases and replaces the four constructors with a family, one per atom shape [1, Def. 3.8, §6]. The rule table here is therefore generated rather than listed, and the counts that depended on there being twelve rules are restated. Section 14 reads phase two from the machine. It reports two new measurements — concurrent erection by the literal construction mixes the apparatus of distinct instances in most runs, and more often under maximal progress than under sequential choice — and proves that the single-address-input discipline, in the form its permutation lemma needs, cannot be met by fixed-arity constructors for any atom mentioning two allocated names. A single-input, multi-hole constructor meets it, erects an instance in two steps, and runs the recursion combinator at constant cost per unfolding. Every compiled unit contains `e`, so the closure results of §6 apply to no compiled program.

Contents

1	What this note is	3
2	The two machines	4
2.1	What a graphics processor rewards	4
2.2	What a tuple-space interpreter does	4
2.3	The gap, and who closes it	4
3	The combinators, and four structural facts	5
4	States are occupation vectors	6
4.1	Names, and the self-indexing problem	7
5	The rule operator	7
5.1	Three classes of rule	8
6	Three invariants, three finiteness conditions	8
6.1	Upper triangularity, precisely	9
7	What actually gets multiplied	9
7.1	Redex finding is a join	10
7.2	Conflict detection is a second product	10
8	The parallel step	11
9	Reflection, at the level of the machine	12
10	Measurements	12
11	Recommendations	13
12	What this does not do	14
13	Obligations	15

14 The two-phase translation, seen from the machine	15
14.1 The signature is generated, and its width is a program property	16
14.2 Every compiled unit sits at the destructor vertex	16
14.3 Erection under maximal progress	16
14.4 The discipline cannot be met by fixed-arity constructors	17
14.5 Single-input context instantiation	17
15 Conclusion	18
A The reference implementation	18

1 What this note is

This note is a design analysis, not a completed theory. It records what the rho combinators of the draft in `F1R3FLY-io/publications/rho-combinators` already give a machine builder, what they do not, and in what order the remaining work should be attempted.

Scope. Classical execution only. Everything here is over the Boolean semiring or the natural numbers. The weighted, complex-valued and quantum readings developed elsewhere in the programme are deliberately absent; they attach to the same operator later, as §12 notes, and nothing below depends on them.

Claims. Six, and nothing more.

- (C1) A combinator state is an occupation vector over a species set, with structural congruence quotiented by construction (§4).
- (C2) The one-step relation is a sum of ladder-operator monomials, one per rule of the generated table, schematic in the names, of degree one more than the largest constructor arity in use (§5).
- (C3) The three invariants of the draft’s lattice section are the three finiteness conditions of that operator, one per lattice axis (§6).
- (C4) Upper triangularity holds exactly on the destructor-free fragment, and the general case needs either the history unfolding or nothing at all (§6.1).
- (C5) Redex finding and conflict detection are sparse matrix products on the atom-name incidence matrix, which is term-sized (§7).
- (C6) Firing a maximal independent set of redexes per step is sound with respect to the interleaving semantics, so machine steps count causal depth (§8).

Not claimed. That any of this beats a well-tuned sequential interpreter on real workloads; the measurements in §10 are on synthetic term families chosen to exhibit known widths, and the width of compiled rho programs is unmeasured. That the reduction graph itself should ever be materialised; the position taken here is the opposite. That maximal progress is the right resolver; it is *a* resolver, and Obligation 13.1 asks what it costs.

Status markers. **SETTLED** for what is proved or measured here or cited from the companion drafts; **ROUTE** for a construction believed to work whose details are not discharged; **OPEN** for a question with no proposed answer; **GATE** for something that must be settled before the work downstream of it is worth starting.

2 The two machines

The mapping asked for is between two concurrency paradigms, so it is worth writing down what each actually is before looking for a morphism.

2.1 What a graphics processor rewards

A modern accelerator is a bulk-synchronous machine with a very wide, very shallow execution model. It rewards, in rough order of importance:

- (G1) **Flat, indexable state.** Arrays of fixed-width records at predictable addresses. Pointer chasing through a heap of variably shaped nodes is the single most expensive thing one can ask of it.
- (G2) **Uniform work across lanes.** Threads in a warp that take the same branch. Data-dependent control flow serialises.
- (G3) **Bulk-synchronous phases.** A kernel launch does one thing to everything, then a barrier. Fine-grained locking and point-to-point synchronisation between work items are not available cheaply.
- (G4) **Standard sparse primitives.** Sparse matrix-vector and matrix-matrix products, segmented scans, sorts, and hash tables have mature, heavily optimised implementations. Work expressed in those terms inherits decades of tuning; work expressed otherwise does not.

2.2 What a tuple-space interpreter does

An RSpace-style interpreter for the ρ calculus does approximately the opposite of all four. Its state is a store of variably shaped terms reached by pointer; matching a receipt against a datum is a recursive walk over nested syntax with binder-sensitive pattern variables; conflicts between competing comm events are resolved by locking or by a single-threaded scheduler; and the inner loop is a hash lookup followed by unification, which is not a sparse linear algebra primitive by any reading.

None of that is a criticism of RSpace, which is a good design for a general machine. It is an observation that the gap between the two paradigms is concrete and itemised, and that a compiler standing between them has a specific job description.

2.3 The gap, and who closes it

The combinators close three of the four gaps by themselves, and the fourth is closed by the step semantics of §8.

$$\begin{array}{ll}
d(a, b, c) \mid m(a, v) \longrightarrow m(b, v) \mid m(c, v) & b_l(a, b) \mid m(a, v) \longrightarrow fw(v, b) \\
k(a) \mid m(a, v) \longrightarrow \mathbf{0} & b_r(a, b) \mid m(a, v) \longrightarrow fw(b, v) \\
fw(a, b) \mid m(a, v) \longrightarrow m(b, v) & s(a, b, c) \mid m(a, v) \longrightarrow fw(b, c) \\
e(a) \mid m(a, v) \longrightarrow *v & fw(a, b) \mid q(a, p) \longrightarrow m(b, @p) \\
cons_l(a, b, c) \mid m(a, @p) \mid m(b, @r) \longrightarrow m(c, @(p \mid r)) & \\
cons_A(a_1, \dots, a_n, f) \mid m(a_1, v_1) \mid \dots \mid m(a_n, v_n) \longrightarrow m(f, @A(v_1, \dots, v_n)) & n = ar(A)
\end{array}$$

Figure 1: The rules, closed under $\mid$ and under the equations, with subjects matched up to $\equiv_{\mathcal{N}}$. The last line is a schema, one rule per member of the constructor family.

Accelerator wants	What supplies it
(G1) flat indexable state	Combinator atoms are fixed-arity tuples of names; under presentation A every argument is a name, hence an integer index (§3).
(G2) uniform work	A rule table generated from the shape table, each rule a fixed shape; one kernel per join plan, no pattern matching at runtime. The pattern matching happened in the compiler.
(G3) bulk-synchronous phases	A step fires a maximal independent set of redexes, which is one barrier per causal layer (§8).
(G4) sparse primitives	Redex finding is a join, conflict detection is a product, independent-set selection is a standard parallel primitive (§7).

The compiler from the ρ calculus to the combinators is therefore load-bearing rather than decorative: it is the component that moves pattern matching from run time to compile time, and it costs only a constant factor in term size (draft 3, Prop. 8.1: the image has $O(|P|)$ atoms).

3 The combinators, and four structural facts

The calculus is that of draft 3, in the repaired form: two sorts, the base atoms

$$m, d, k, fw, b_l, b_r, s, e, q,$$

the parallel constructor $cons_l$, and a constructor $cons_A$ of arity $ar(A) + 1$ for each atom shape A that a program needs [1, Def. 3.8]; the equations of an abelian monoid on $\mid$ together with $@*a = a$, and, under presentation A, $*@p = p$. The rules are reproduced in Figure 1, the constructors as one schema.

Four facts about this presentation do the work of the rest of the note. Three are theorems of the companion drafts; the fourth is a design recommendation.

- (F1) **Atoms are flat.** Every atom is a head symbol applied to at most w arguments, and every argument is a name, except the payload of q and, under presentation B, of m . Here w is the largest arity in the family a program uses: four for the static-apparatus signature, five once phase two must erect the constructors of (o5) chains (§14); it is known at compile time. There is no nesting, no binder, and no pattern variable.

- (F2) **The multiset is complete.** By the logic note's component lemma, $\lfloor P \rfloor$, the multiset of top-level atoms of the drop-free normal form, is a complete invariant: $P \equiv Q$ iff $\lfloor P \rfloor = \lfloor Q \rfloor$ as multisets. **SETTLED**
- (F3) **Rules are small and subject-joined.** Every rule has between two and w premises, each premise a single atom, and the premises share names. The logic note remarks that this is the only structural fact its modality uses; it is also the only structural fact the machine below uses. **SETTLED**
- (F4) **Name equality is decidable and internable.** On closed drop-free normal forms $\equiv_{\mathcal{N}}$ is decidable, by normalisation followed by multiset comparison, and $@-$ is injective there. So names can be hash-consed to integers once and compared by integer equality thereafter. **SETTLED**

Remark 3.1 (Presentation A is the machine presentation). Draft 3 leaves the choice between presentations A and B open on grounds of where decoding happens. For a machine the choice is forced, and in favour of A. Under A a message payload is a name, hence an integer, so every atom is a fixed-width tuple of integers and (G1) is satisfied exactly. Under B a payload is a process, so an atom is a tuple of integers together with a variable-sized block, and the store becomes ragged.

The same move should be made for $\mathbf{q}$. Although its payload is sorted as a process, storing it by its quote — holding $\mathbf{q}(a, p)$ as the pair $(a, @p)$ — loses nothing, because $@-$ is injective on normal forms by (F4), and it makes the **quote** rule a field copy rather than a construction. This is the difference between rule class (a) and class (b) in §5, and it is obtained for free. **SETTLED**

The same choice makes the constructor family uniform. Draft 3 states the member for $\mathbf{q}$ in presentation B's sorting and notes that under A it must be written $@q(v_1, *v_2)$, relying on the equation [1, Rem. 3.9]. With the payload of $\mathbf{q}$ held by its quote, the normal form of that name is the row $(\mathbf{q}, v_1, v_2)$, so $\mathbf{cons}_{\mathbf{q}}$ is an ordinary member whose conclusion is built from its premise names like any other, and the coercion never appears in the machine.

4 States are occupation vectors

Definition 4.1 (Species). A *species* is a pair of a head symbol and a tuple of names of the matching arity: the set is $\mathcal{S} = \{(A, \vec{n}) : A \text{ a head symbol}, \vec{n} \in \mathcal{N}^{\text{ar}(A)}\}$, where $\mathcal{N}$ is the set of names modulo $\equiv_{\mathcal{N}}$.

By (F2) a state is a finite multiset over $\mathcal{S}$, that is, a function $\gamma : \mathcal{S} \rightarrow \mathbb{N}$ of finite support. Write $\mathbb{F}$ for the free module on the set of such γ and $|\gamma\rangle$ for the basis vector at γ . Equivalently, and more usefully, $\mathbb{F}$ is the free commutative algebra generated by $\mathcal{S}$: a state is a monomial $\prod_{\sigma} (a_{\sigma}^{\dagger})^{\gamma(\sigma)} |\mathbf{0}\rangle$, and the abelian monoid equations of the calculus are the commutativity of that product.

Principle 4.2 (Structural congruence is free). The equations of Definition 3.2 of draft 3 are not imposed on the representation and are not discharged by a normalisation pass. Commutativity, associativity and the unit of $|$ are the algebra of the multiset; $*@p = p$ and $@*a = a$ are discharged once, at interneting time, by the drop-elimination lemma. No run-time cost. **SETTLED**

This is the first place where the combinators pay. In a direct **rho** machine, matching modulo structural congruence is a recurring cost and a recurring source of subtlety; here it is a property of the data structure.

4.1 Names, and the self-indexing problem

$\mathcal{S}$ is indexed by $\mathcal{N}$, and $\mathcal{N}$ is the set of quoted processes, so by (F2) it is the set of finite multisets over $\mathcal{S}$. The index set of the basis therefore satisfies

$$\mathcal{N} \cong M_{\text{fin}}(\mathcal{S}), \quad \mathcal{S} = \coprod_A \mathcal{N}^{\text{ar}(A)}.$$

The basis of the state space indexes itself. This is reflection, seen through the linear algebra, and it is the reason no finite matrix can be written down in advance for the full calculus. The solution of the domain equation is the initial algebra: names are finite trees, so $\mathcal{N}$ is countable, finitely branching, and graded by depth of quotation.

Definition 4.3 (The intern table). The machine realises $\mathcal{N}$ as a bijection between integers and canonical component multisets, in two directions:

$$\begin{aligned} \text{quote} : M_{\text{fin}}(\mathcal{S}) &\rightarrow \mathbb{N} && \text{hash-cons; an encode,} \\ \text{drop} : \mathbb{N} &\rightarrow M_{\text{fin}}(\mathcal{S}) && \text{table read; a decode.} \end{aligned}$$

Remark 4.4 (Interning order is a gauge freedom). Two runs that fire the same rules in different orders may assign different integers to the same name, because integers are handed out on insertion. The resulting states are equal up to the canonical bijection determined by **quote**, which is injective. Nothing observable depends on the assignment, and the machine must never let an integer comparison stand in for a semantic distinction between runs. **SETTLED**

5 The rule operator

Definition 5.1 (Ladder operators). For $\sigma \in \mathcal{S}$, let $a_\sigma^\dagger$ add one occurrence of σ and a_σ remove one, with $a_\sigma|\gamma\rangle = 0$ when $\gamma(\sigma) = 0$.

Definition 5.2 (The transfer operator). For a rule r with premise multiset L_r and conclusion multiset R_r , both over atom shapes with name variables, and for a substitution θ of those variables by names,

$$H_r = \sum_{\theta} \left(\prod_{\beta \in R_r \theta} a_\beta^\dagger \right) \left(\prod_{\alpha \in L_r \theta} a_\alpha \right), \quad T = \sum_r H_r.$$

This is the factorisation asked for. T is a representation of the rewrite rules alone: it is schematic in the names, independent of any particular term, and of degree at most w by (F3). The unfolding of the reduction graph is repeated application of T to $|\gamma\rangle$.

Proposition 5.3 (Matrix elements count derivations). $\langle \gamma' | H_r | \gamma \rangle$ is the number of distinct occurrence tuples in γ matching L_r and producing γ' , namely $\prod_{\alpha} (\gamma(\alpha))_{\underline{m}_\alpha} / \prod_{\alpha} m_\alpha!$ where m_α is the multiplicity of α in $L_r \theta$ and $(n)_{\underline{m}}$ is the falling factorial.

Proof. Standard for multiset rewriting: a match is a choice of m_α distinct occurrences of each premise species, unordered within a species. $\square$

Remark 5.4 (This is mass action, and it is not an accident). Proposition 5.3 is the mass-action law, and the coefficient is the same multiplicity h that the Gillespie implementation in **rho-life** already computes. The combinators are a chemical reaction network whose reaction schemes are generated from its shape table. The classical reading taken here uses only the Boolean shadow of that coefficient; the weighted readings elsewhere in the programme use the coefficient itself.

5.1 Three classes of rule

The rules are not alike in how much of T can be written down ahead of time.

Class (a), seven rules: permutation of existing indices.

dist, **kill**, **forward**, **bind_l**, **bind_r**, **synch**, **quote**. The conclusion is a fixed multiset of atoms whose names all occur in the premise. H_r is a constant-coefficient ladder monomial, fully schematic, requiring no access to the intern table at all. Note that **quote** belongs here only because of Remark 3.1.

Class (b), one rule per constructor: encode.

The constructor family. The conclusion is a single atom, so the arity is fixed, but its payload is a *new* index, obtained by hash-encoding a term built from the premise names. H_r grows the index set.

Class (c), one rule: decode.

eval. It is the only rule whose conclusion multiset has a size depending on the *value* of a premise name. Its rows cannot be written schematically; they require reading the intern table.

Principle 5.5 (Where the laziness is, and which kind). The matrix must be materialised lazily for two distinct reasons, and the lattice separates them. Class (b) makes the *basis* generate on demand: new columns appear. Class (c) makes the *rows* generate on demand: an existing column's image is not known until the index is decoded. Class (a), which is most of the calculus, is fully eager. **SETTLED**

6 Three invariants, three finiteness conditions

Section 9 of draft 3 proves three invariants in order to draw the expressiveness lattice. They are, without modification, the three finiteness conditions a matrix representation needs, and each is governed by a different axis of the lattice. This is the note's main structural observation.

Theorem 6.1 (The lattice classifies the matrix). *Let T be a combinator term over a constructor set S .*

- (i) (Index set; constructor axis.) *By the name-closure lemma, every name occurring in a reduct of T lies in the closure of $\mathcal{N}^*(T)$ under the node shapes $\{A : \text{cons}_A \in S\}$. If $S = \emptyset$ the reachable name set is $\mathcal{N}^*(T)$, which is finite, so $\mathcal{S}$ restricted to reachable species is finite of size $O(|\mathcal{N}^*(T)|^w)$. If $S \neq \emptyset$ the reachable name set is the free S -algebra on $\mathcal{N}^*(T)$, countable and graded by depth.*
- (ii) (Nilpotency; destructor axis.) *By the atom-count lemma every rule but **eval** strictly decreases the weight W , which counts non-message atoms with $\mathbf{b}_l$, $\mathbf{b}_r$ and $\mathbf{s}$ counted twice. Without $\mathbf{e}$, the function $\gamma \mapsto W(\gamma)$ is a grading strictly decreased by every step, so the reachable sub-operator is strictly block upper triangular with respect to it and $T^{W(T)+1} = 0$. Since $W(T)$ is at most twice the number N of non-message atoms of T , also $T^{2N+1} = 0$; the index $N+1$ does not suffice, as $\mathbf{b}_l(a, b) \mid \mathbf{m}(a, v) \mid \mathbf{m}(v, x)$, with $N = 1$ and a derivation of length two, shows.*
- (iii) (Occupation bound; duplication axis.) *By the message-count lemma **d** is the only rule that increases the number of messages without consuming a $\mathbf{q}$ or opening a payload. The **quote** rule does increase the number of messages, by one, while consuming a $\mathbf{q}$; so without **d** and without **e** it is the number of messages and stores together that is non-increasing, and the combined occupation of $\mathbf{m}$ and $\mathbf{q}$ species is bounded by that of T . The message count alone is not monotone: $\mathbf{fw}(a, b) \mid \mathbf{q}(a, p) \longrightarrow \mathbf{m}(b, @p)$ raises it from 0 to 1.*

Proof. Each clause is the cited lemma read as a statement about the operator. For (ii), a strictly decreasing integer grading bounded below by 0 and above by $W(T)$ gives a block decomposition in which T maps block j into blocks $< j$, which is the definition of strict block upper triangularity, and composition of $W(T) + 1$ such maps is zero. For (iii), inspection of the rules: without **d** and **e**, **forward** preserves the number of messages and stores, **quote** exchanges a store for a message, and every other rule decreases it. $\square$

Corollary 6.2 (Closed-form reachability on the destructor-free fragment). *Without **e**, the reachability closure is the terminating Neumann series $\sum_{k \leq W(T)} T^k = (\mathbf{1} - T)^{-1}$, computable as the product $\prod_{i < \lceil \log_2(W(T)+1) \rceil} (\mathbf{1} + T^{2^i})$ of $\lceil \log_2(W(T) + 1) \rceil$ factors obtained by repeated squaring, each a sparse matrix product. SETTLED*

Remark 6.3 (One atom, three characterisations). The destructor **e** is, simultaneously: the carrier of Turing power, by the draft’s strong-normalisation proposition; the unique breaker of the grading, by Theorem 6.1(ii); and the unique rule whose operator cannot be written schematically, by §5.1. Three independent characterisations of the same atom, arrived at from logic, from order theory and from implementation. The draft’s remark that “the destructor carries the computational power that reflection supplies” acquires an operational reading: the power is exactly the ability to turn an index into a state.

Remark 6.4 (The Petri net reading). Without constructors and without **e**, the combinators are a place/transition net: places are species, transitions are rule instances, and the step semantics of §8 is the net’s step semantics. Adding constructors makes the place set generated rather than given; adding **e** makes the transition relation depend on the contents of a place. The lattice is therefore also a descent from nets through generated-place nets to Turing completeness, which is a second reading of the same three invariants and may be worth a paragraph in the draft. ROUTE

6.1 Upper triangularity, precisely

The proposal that opened this line of work was to encode the reduction graph as an upper triangular matrix. Upper triangularity is equivalent to acyclicity of the graph, and the reduction graph of a combinator term is not acyclic in general: reflection admits terms that step to a structurally congruent copy of themselves, and by Principle 4.2 structurally congruent means equal. So on the nose the encoding is available exactly on the destructor-free fragment, by Theorem 6.1(ii).

Two repairs are available, and for the classical execution goal the second is recommended.

- (1) **Work in the unfolding.** The free history of a term is the universal cover of its reduction graph, graded by step count, hence acyclic by construction; triangularity is then free, at the cost of never identifying a returning state. The cycles one gives up are exactly the non-trivial elements of π_1 that the history endofunctor’s erasure lattice already parametrises, so the apparatus exists. This is the right route if the closure is the object wanted, for instance for model checking. ROUTE
- (2) **Do not ask for it.** For running a program one needs a frontier, not a closure: the current state and its successors. Triangularity is a property of the whole graph and buys nothing at the frontier. A general sparse non-negative operator applied to a sparse vector is entirely adequate, and iterative methods are indifferent to cycles. SETTLED

7 What actually gets multiplied

There are three matrices in play and conflating them is the easiest mistake to make here.

	Indexed by	Size, and what it is for
T	markings $\times$ markings	Astronomically large, never materialised. The semantics.
B	redexes $\times$ occurrences	$O(\text{redexes})$ nonzeros. Conflict detection.
A	atom occurrences $\times$ names	Term-sized , exactly $\sum_{\text{atoms}} \text{ar}(A)$ nonzeros. The thing the machine multiplies.

Definition 7.1 (Incidence). For each head symbol A and each argument slot $j < \text{ar}(A)$, let M_j^A be the Boolean matrix with $M_j^A[i, n] = 1$ iff the i -th occurrence of A carries name n in slot j . Each row has exactly one nonzero; the whole family has $\sum_{\text{atoms}} \text{ar}(A) \leq 4|\gamma|$ nonzeros.

7.1 Redex finding is a join

By (F3) every rule is a conjunction of atom premises sharing names, which is a relational join on the incidence matrices. Writing $\cdot$ for Boolean sparse matrix product:

Rules	Expression	Cost
the seven two-premise rules against m quote	$M_0^C \cdot (M_0^m)^\top$	one SPGEMM; nonzeros $\sum_n c_n \mu_n$ where c_n, μ_n are the numbers of consumers and messages at n
cons_I , and cons_A for A binary	$M_0^{\text{fw}} \cdot (M_0^q)^\top$ $X = M_0^{\text{cons}} \cdot (M_0^m)^\top$, $Y = M_1^{\text{cons}} \cdot (M_0^m)^\top$, then the row-wise Cartesian product $X \bowtie Y$ with $j \neq k$ masked out	as above two SPGEMMs plus a segmented expansion
cons_A for A of arity $n \geq 3$	n such products, then an n -fold row-wise product with distinctness masking	n SPGEMMs plus expansion

Remark 7.2 (Contention is a join cost, not a scheduling cost). The two-premise product has $\sum_n c_n \mu_n$ nonzeros, which is quadratic in the occupancy of a contended channel. This is real and is measured in §10 (experiment E2, where c consumers and c messages on one channel produce c^2 redexes). It does *not* cost parallelism: the maximal independent set on that quadratic redex set is a perfect matching of size c , found in one step. The cost is enumeration, and the standard remedy — worst-case-optimal join, or bucketing by name before the product — applies unchanged. OPEN which remedy is right at scale.

7.2 Conflict detection is a second product

Definition 7.3 (Conflict). Two redexes conflict if they consume a common occurrence. With B the redex-occurrence incidence matrix, the conflict graph is the off-diagonal nonzero pattern of $B \cdot B^\top$.

So the entire per-step analysis — which redexes exist, and which of them may fire together — is two rounds of Boolean sparse matrix multiplication on term-sized operands. That is the concrete content of “rho execution as linear algebra” in the classical setting, and it involves no matrix whose dimension is the number of states.

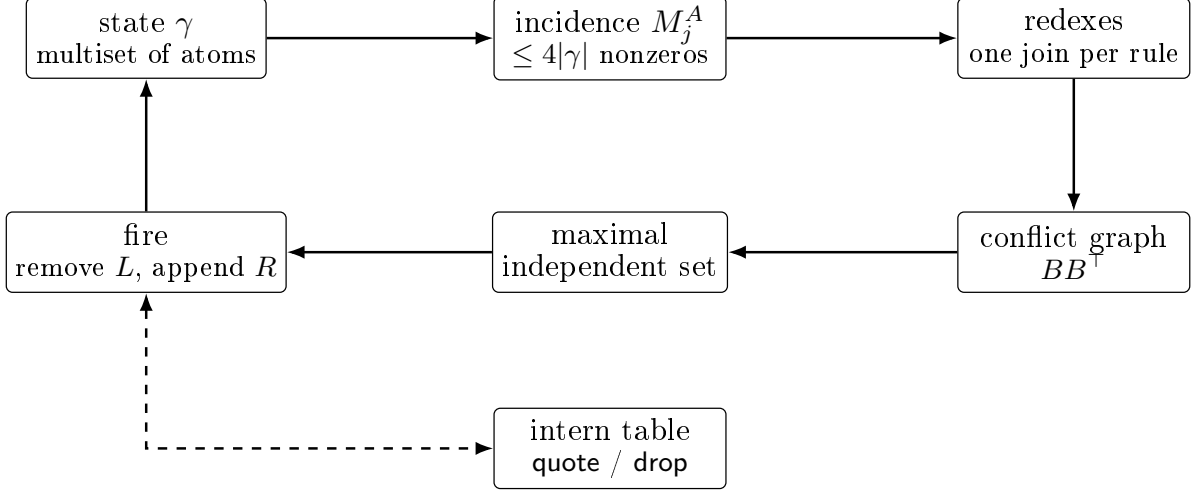


Figure 2: One machine step. Every solid arrow is a data-parallel bulk operation. The dashed arrow is the encode and the decode of §9, used only by rule classes (b) and (c).

8 The parallel step

This section supplies (G3), and is where the concurrency of the calculus meets the concurrency of the machine.

Definition 8.1 (Independence, step). Redexes r_1, r_2 enabled in γ are *independent* if the occurrence multisets they consume are disjoint. A set S of pairwise independent redexes is a *step*; $\gamma \Rightarrow_S \gamma'$ removes every occurrence consumed by a member of S and adds every atom produced by a member of S .

Lemma 8.2 (Diamond). *If r_1, r_2 are independent and enabled in γ , then each remains enabled after the other fires, and both orders yield the same state.*

Proof. Firing removes only the occurrences consumed, so by disjointness the premises of r_1 survive the firing of r_2 and conversely. The produced multiset of r_i is a function of the occurrences r_i consumes and of the intern table; by Remark 4.4 the table's contribution is order-independent up to the canonical bijection. Both orders therefore give $\gamma - L_{r_1} - L_{r_2} + R_{r_1} + R_{r_2}$. $\square$

Theorem 8.3 (Step soundness). *If $\gamma \Rightarrow_S \gamma'$ then $\gamma \longrightarrow^* \gamma'$ by a sequential derivation firing exactly the members of S , in any order, and all such orders agree.*

Proof. Induction on $|S|$ using Lemma 8.2. $\square$

Corollary 8.4 (Steps count causal depth). *A parallel run of k steps performs $\sum_i |S_i|$ rewrites. The available speedup over a sequential interpreter is therefore the mean step width, and the number of machine steps is bounded below by the causal depth of the derivation.*

Remark 8.5 (Maximal progress is a resolver, not a theorem). The converse of Theorem 8.3 fails, and deliberately. Requiring S maximal excludes interleavings in which an enabled, unconflicted redex is left unfired; and when redexes do conflict, the choice of which enters S is precisely the choice a scheduler makes. In the vocabulary of the programme's semantics work, maximal progress is a *resolver* for the calculus's non-determinism, of the same kind as physical core races in an RSpace implementation, and not the same resolver. It is also the true-concurrency reading: a maximal-progress machine observes a partial order of causes rather than a sequence, which is what the programme has elsewhere argued the rewrites should determine. See Obligation 13.1.

Remark 8.6 (Contention sets, generalised). The conflict structure is a hypergraph, since rules have up to w premises, so the object here is an independent set in the conflict graph of redexes rather than a matching in a bipartite graph. This is the k -ary generalisation of the contention-set normalisation and maximal matchings developed for the graded where-clause work, and the two should be reconciled before either is implemented twice. **GATE**

Remark 8.7 (Selection is a standard primitive). A maximal independent set with random priorities is a classical parallel algorithm running in $O(\log n)$ expected rounds, and re-randomising the priorities each step makes starvation of any given redex have probability tending to zero. The reference implementation uses the naive greedy version; a device implementation should use the standard one.

9 Reflection, at the level of the machine

Rule classes (b) and (c) are the two lazy parts of the operator. It is worth saying plainly what they cost, because the answer is surprising.

Decode, the eval rule.

$e(a) \mid m(a, v) \longrightarrow *v$ reads the intern table at v and appends the stored component multiset to the state. On a device this is a table lookup followed by a bulk copy of a contiguous block: among the cheapest things an accelerator does.

Encode, the constructors.

$\text{cons}_m(a, b, c) \mid m(a, u) \mid m(b, v) \longrightarrow m(c, @m(u, v))$ builds a small canonical term and hash-conses it. Concurrent hash tables with insert-or-get semantics are a standard device primitive.

Principle 9.1 (Reflection is an intern table). The feature that makes the calculus theoretically hard — that a name is a program and a program can be recovered from a name — is, at the level of the machine, exactly a bidirectional intern table. The theoretical difficulty and the implementation difficulty do not coincide here, and they point in opposite directions: reflection is what obstructs a static matrix and what makes the dynamic one cheap, because storing a continuation as an index is what keeps atoms fixed-width.

This also explains the succinctness result of draft 3 from the machine side. Yoshida’s elimination must rewrite a continuation to delay it, costing $\Theta(3^d)$ in nesting depth; the reflective encoding stores it as a name and pays for a gate. A machine that stores continuations as integers in a table is doing the same thing, and the $O(|P|)$ bound on the encoding is what keeps the state array small enough to be worth the transfer to a device.

Remark 9.2 (Two serialisation points). The intern table is shared mutable state and is therefore the one place the bulk-synchronous discipline is violated. Both operations are commutative and idempotent in effect, by Remark 4.4, so the table can be updated by an unordered batch insert per step rather than by locking. Whether the table should also be garbage collected, and on what criterion, is Obligation 13.3.

10 Measurements

A reference implementation accompanies this note (`rhocombmat.py`, `experiments.py`). It represents a state as a multiset of atom occurrences, builds the incidence matrices of Definition 7.1 with `scipy.sparse`, finds redexes by the products tabulated in §7, builds the conflict graph as $BB^\top$, selects a maximal independent set, and fires it. It is a correctness reference, not a performance artefact; the numbers below are step counts and widths, not timings.

Empirical finding 10.1 (The join formulation is correct). On 400 randomly generated states over every shape of the generated table — in version 1.2, two levels of the family and the context constructor of §14, twenty-nine shapes — and two to five distinct names, the redex set computed by sparse matrix products agrees exactly with independent naive enumeration: 400/400.

Empirical finding 10.2 (Steps count width). On four term families, averaged over five seeds:

Term	atoms	par. steps	rewrites	seq. steps	max width	ratio
E1 fan, $k = 1, d = 8$	9	8.00	8	8	1	1.00
E1 fan, $k = 4, d = 8$	36	8.00	32	32	4	4.00
E1 fan, $k = 16, d = 8$	144	8.00	128	128	16	16.00
E1 fan, $k = 64, d = 8$	576	8.00	512	512	64	64.00
E2 contended, $c = 4$	8	1.00	4	4	4	4.00
E2 contended, $c = 16$	32	1.00	16	16	16	16.00
E3 duplicator tree, $h = 2$	8	3.00	7	7	4	2.33
E3 duplicator tree, $h = 4$	32	5.00	31	31	16	6.20
E3 duplicator tree, $h = 6$	128	7.00	127	127	64	18.14
E4 construct + eval, $k = 1$	4	2.00	2	2	1	1.00
E4 construct + eval, $k = 8$	32	2.00	16	16	8	8.00
E4 construct + eval, $k = 32$	128	2.00	64	64	32	32.00

E1 is k independent forwarder chains of length d : parallel steps are exactly d , independently of k , and the ratio is exactly k . E2 is c messages and c consumers on a single channel: one parallel step, at the cost of c^2 enumerated redexes (Remark 7.2). E3 is a binary duplicator tree of height h : parallel steps are $h + 1$ against $2^{h+1} - 1$ sequential steps. E4 exercises rule classes (b) and (c) together, building a name with cons_m and opening it with e .

Remark 10.3 (What these do and do not establish). They establish that the construction is correct and that the step count is the causal depth on families whose depth is known analytically. They establish nothing about real ρ programs, because the width of the combinator image of a compiled ρ term has not been measured. That measurement is the first item in §11 for a reason: it is the gate on the whole enterprise, and it requires the compiler, not the machine. GATE

Principle 10.4 (The width thesis). The speedup available from this route is the mean width of the reduction, and nothing else. Sequential terms gain nothing; the machine adds a constant per-step overhead of two sparse products and an independent-set selection. The thesis is falsifiable by measuring the width distribution of compiled ρ programs, and should be falsified or confirmed before any device work is undertaken.

11 Recommendations

Seven phases, each with an exit criterion. Phases P0–P2 are host-side and establish whether the enterprise is worth pursuing; P3 onward are the device work and should not begin until P2’s number is in hand.

P0. Presentation and storage decisions.

Adopt presentation A; store the payload of $\mathbf{q}$ by its quote (Remark 3.1). Build the intern table with canonical multiset hashing of names. *Exit*: $\equiv_{\mathcal{N}}$ decided by integer comparison on a corpus of terms, cross-checked against structural comparison.

P1. The compiler, instrumented.

Complete the `rho-to-combinators` compiler already scoped in the `rho-combinators` programme, and instrument it to emit the atom multiset directly in the machine’s representation. *Exit:* the $O(|P|)$ bound observed on a corpus; round-trip against a `rho` interpreter on a behavioural test suite.

P2. The width measurement.

Run the host reference machine on the compiled images of real programs and record the width distribution per step. *Exit:* a number. Principle 10.4 stands or falls here. **GATE**

P3. Host implementation in earnest.

Port the reference machine to Rust against a sparse Boolean backend, one kernel per rule, with bucketing by name ahead of the two-premise products. *Exit:* differential agreement with the sequential interpreter on reachable-state sets for the destructor-free fragment, where Corollary 6.2 gives an independent closed form.

P4. The device port.

Move the incidence matrices, the join kernels, the conflict product and the independent-set selection to the accelerator; keep the intern table host-side initially, then move it behind a batched insert-or-get. *Exit:* identical step traces to P3 under a fixed priority seed.

P5. Contention remedies.

Replace the naive two-premise product with a worst-case-optimal join or name-bucketed product where profiling shows $\sum_n c_n \mu_n$ dominating. *Exit:* enumerated redexes per step within a constant factor of fired redexes on the P2 corpus.

P6. The by-product for the logic.

On the destructor-free fragment, expose $(\mathbf{1} - T)^{-1}$ by repeated squaring, and use it as the model checking engine for the spatial-behavioural logic of the companion note, where the modality is a matrix–vector product and the fixed-point operators are iterations of it. *Exit:* agreement with a direct model checker on the lattice-occupancy formulae.

Remark 11.1 (Why P6 is not an afterthought). The transfer operator’s natural consumer is the logic, not the interpreter. The modality $\langle K \rangle \varphi$ is a sum over successors, which is $T^\top$ applied to the characteristic vector of φ , and the $\mu X(\vec{a}).\varphi$ operator is an iteration of that. The companion note’s requirement that the lattice-occupancy formulae be decidable and Theorem 6.1’s nilpotency condition are plausibly the same condition seen twice; establishing that would be a genuine result and costs little once P3 exists. **ROUTE**

12 What this does not do

Stated plainly, so that no one is disappointed later.

- **No asymptotic improvement to a single sequential run.** The total number of rewrites is unchanged. Only their scheduling changes.
- **No help for narrow terms.** A term whose reduction is a chain gains nothing and pays the per-step overhead.
- **No reachability decision procedure.** Reachability remains undecidable in the presence of **e**; the operator is locally finite, so finite iteration is always well defined, but the closure need not be reached.

- **No treatment of weights.** Everything here is Boolean. The weighted reading attaches at exactly one place — a diagonal factor D_r in H_r whose entries depend on the premise names — and the combinators, having no where-clause, would need the weights supplied per rule instance rather than per spatial formula. That is a separate note.
- **No claim about per-copy identity.** Because $\equiv$ is multiset equality, two occurrences of the same atom are genuinely indistinguishable. Machine-level occurrence identities exist only to define conflict and must not leak into the semantics. Any downstream construction that wants per-derivation identity has to get it from somewhere else.

13 Obligations

Obligation 13.1 (Resolver fidelity). Maximal progress with re-randomised priorities induces a distribution over runs. Characterise it, and say how it differs from the distribution induced by uniform choice of a single redex. The question matters because the programme elsewhere treats the resolver as a parameter of the interpretation, and a machine that silently fixes one has made a semantic commitment. **OPEN**

Obligation 13.2 (Canonical name hashing). $\equiv_{\mathcal{N}}$ is decidable by recursive multiset comparison, but the machine wants a canonical hash computable bottom-up in one pass over the name’s tree, stable across runs. Give one, and bound its collision behaviour on the corpus. **ROUTE**

Obligation 13.3 (Garbage). Inert atoms accumulate: a $\mathbf{k}$ with no message, a message with no consumer, a $\mathbf{q}$ nobody quotes. They cost incidence nonzeros on every step. Decide whether inertness is detectable cheaply — a species with no possible partner under any rule is detectable by a name-degree test — and whether the intern table needs collection at all. **OPEN**

Obligation 13.4 (Contention set reconciliation). Reconcile the independent-set construction of §8 with the contention-set normalisation of the graded where-clause work, which solves the binary case. One of the two should be derived from the other. **GATE**

Obligation 13.5 (Occupancy decidability and nilpotency). Determine whether the companion logic note’s requirement that lattice-occupancy formulae be decidable coincides with the finiteness conditions of Theorem 6.1. **ROUTE**

Obligation 13.6 (Fairness). Prove or refute that re-randomised-priority maximal progress is fair, in the sense that a continuously enabled redex fires with probability one. **OPEN**

Obligation 13.7 (Width of real programs). Measure the width distribution of the combinator images of compiled rho programs. This is P2 and it gates everything downstream. **GATE**

14 The two-phase translation, seen from the machine

Draft 3 now compiles in two phases [1, §6]. Phase one targets the combinators extended with a name binder. Phase two removes the binder: each run-time instance receives an *address* σ at a static channel r , allocates its apparatus beneath it as leaves built by $\mathbf{L}\sigma = @m(\sigma, @0)$ and $\mathbf{R}\sigma = @m(\sigma, K)$, and erects each atom that mentions an allocated name by a constructor followed by $\mathbf{e}$ [1, Rem. 3.11, Def. 6.8]. The repair is correct in purpose — positional apparatus names are unsound for code instantiated twice [1, §6.1] — and it changes what the machine sees in five ways. The reference implementation is extended accordingly: its rule table is generated from a shape table, and `phase2.py` runs the experiments below.

14.1 The signature is generated, and its width is a program property

The rule table is one row per shape, and the constructor for a shape is derived from it. Phase two erects every phase-one atom that mentions a bound name, and the (o5) chains of phase one consist of constructor atoms whose channels are bound names, so a program with an (o5) occurrence needs $\text{cons}_{\text{cons}_d}$ and its siblings, of arity five with four premises. The family is finite for each program [1, Rem. 6.15], but it is not the one-level family of base shapes, and the fixed row width of Remark 3.1 is therefore a compile-time parameter w read off the program rather than a constant of the calculus. This is why (F1), (F3), claim (C2) and Theorem 6.1(i) are now stated in terms of w .

14.2 Every compiled unit sits at the destructor vertex

Each translated input stores its body behind a gate containing e [1, Lem. 6.2], and each erection is a constructor followed by e . Every compiled program with an input, and every phase-two image, is therefore at the top of the computation axis. Theorem 6.1(ii), Corollary 6.2 and phase P6 of §11 apply to hand-written combinator terms and to input-free programs, and to no compiled program with an input. The frontier reading of §6.1, route (2), is the only one available to compiled code, which is what the machine of §8 uses in any case.

The name universe also grows with every unfolding, since each instance allocates beneath a strictly deeper address [1, Ex. 6.9]. With names interned structurally, as the reference does, a leaf beneath σ is one new table entry whose single component refers to σ 's index, so descending one level costs $O(1)$ whatever the depth; a content hash of the full encoding would cost $O(\text{depth})$ per new address. This is a reason, beyond those of §9, for the machine not to hash on its hot path.

14.3 Erection under maximal progress

Two deploys of one program, or two unfoldings of one recursive unit, are two instances sharing every static channel of the phase-two image. Draft 3 makes this safe under the single-address-input discipline [1, Def. 6.10, Lem. 6.11], and the literal erection of [1, Def. 6.8] does not satisfy it: $\text{cons}_A(t_{i_1}, \dots, t_{i_m}, f_A)$ has m address inputs at static channels. The question for a bulk-synchronous machine is how often that bites.

Empirical finding 14.1 (Concurrent literal erection mixes instances). Take the unit $\{\text{fw}(z_1, z_2), \text{k}(z_1)\}$, compiled by [1, Def. 6.8] with per-node gadget channels and a leaf fan-out for z_1 , and deploy n instances at distinct fresh addresses. An erected forwarder is *mixed* if its two leaves lie beneath different addresses. Over 50 seeds:

instances	maximal progress		sequential uniform			
	runs	mixed	forwarders	mixed	runs	mixed
2		29/50	58		16/50	32
4		48/50	144		38/50	95
8		50/50	347		49/50	268
16		50/50	754		50/50	659
						erected
						100
						200
						400
						800

With single-input context instantiation (§14.5) no forwarder is mixed in any run, at any instance count, under either scheduler; erection takes two parallel steps against five.

The fraction of mixed forwarders approaches $1 - 1/n$, which is what uniformly random pairing of left and right leaves would give. Maximal progress mixes more than sequential choice because it fires every enabled constructor in the same step, when both instances' leaves are present at the static channels together. A device running many instances is the worst case, not a corner case: correctness cannot be left to the schedule.

14.4 The discipline cannot be met by fixed-arity constructors

Draft 3's remedy is a conforming construction [1, Rem. 6.12]: thread one copy of the address through single-input chains and perform every join at an address-derived channel. The following says that no such construction exists for the atoms phase two most needs.

Lemma 6.11's proof turns on one property of a redex fired at a static channel: that it cannot join material from two instances. Draft 3 states this for constructors, but a router does the same when it holds a leaf and receives an address — $\text{fw}(k, \text{L}\sigma) \mid \text{m}(k, \text{L}\tau)$ at static k delivers τ 's leaf into σ 's apparatus — so the form of the discipline the lemma actually uses is the following.

Definition 14.2 (The discipline, as the permutation lemma uses it). Fix an instance at address σ . A redex *joins at a static channel* if one of its consumed channels is a static name and two of its premises contribute occurrences of σ to its produced atoms. An image satisfies (D^*) if no reachable redex joins at a static channel.

Proposition 14.3 (Address multiplicity). *Under (D^*) , with the rules of [1, Def. 3.3, Def. 3.8], no reachable atom contains two occurrences of σ . Hence no atom mentioning two leaves beneath σ is ever erected, and neither is any phase-one atom mentioning two bound names — the distributor's $\text{d}(q_1, p_1, q_2)$ and the gate's $\text{s}(p_0, b, c)$ among them.*

Proof. Induction on reduction. Initially the phase-two image is closed and contains no occurrence of σ , and the address arrives as $\text{m}(r, \sigma)$, one occurrence. Suppose every atom so far contains at most one. A message or a store is an atom, so one whose payload contains σ has a subject free of σ , which is a static channel. Every rule builds its produced atoms from names of its premises — the consumer's arguments and the payloads of the consumed messages or store — together with static names and, for **eval**, the components of one payload, which contain at most one occurrence between them. A produced atom with two occurrences must therefore draw one from each of two premises. Those are two payloads, or a consumer argument and a payload; in either case a payload carrying σ is consumed, so a consumed channel is static, and the redex joins at a static channel, which (D^*) excludes. A leaf beneath σ contains exactly one occurrence of σ , so an atom mentioning two leaves contains two. $\square$

A fixed-arity constructor places each input once. To build a name mentioning σ twice from one copy of σ it must duplicate and rejoin, and the rejoin is the forbidden join. The context-indexed constructor of [1, Rem. 3.14], with holes in name positions, places one input at every hole. Draft 3 records that it is derivable from the family by chaining, and that the two have the same reach; Proposition 14.3 says the derivation does not preserve (D^*) , so under the discipline the two do not have the same reach, and phase two needs the context-indexed form as a primitive. SETTLED for the proposition; the choice of primitive is the paper's.

14.5 Single-input context instantiation

Write $\text{inst}(a, f, C) \mid \text{m}(a, v) \longrightarrow \text{m}(f, @C[v])$ for a context C with a hole in name positions, possibly many, including beneath quotation. A unit's phase-two image becomes two atoms,

$$\text{inst}(r, f, C_{\text{unit}}) \mid \text{e}(f),$$

where C_{unit} is the phase-one image with each bound name z_i replaced by the leaf at a fixed-length path w_i beneath the hole, and with the spare leaves posted for nested units and for self-instantiation. The constructor at the static channel r has one premise, so (D^*) holds by construction; permuting addresses among **inst** atoms is the harmless permutation of Lemma 6.11; and the whole instance appears in two steps. The allocation tree, the per-node gadget channels, the fan-out count and the constructors of constructors all disappear, and w returns to four.

For the machine, `inst` is one more class-(b) rule with a single premise: its row is fixed-width, its join is the cheapest there is, and its encode is a substitution over the context’s interned form, memoised per name, costing $O(|C|)$ new table entries per instance. The hole is a reserved index that no rule can construct.

Empirical finding 14.4 (The recursion combinator at constant cost). $D_x = \text{for}(y \leftarrow x)(x!(\ast y) \mid \ast y)$ compiled with `inst`, per [1, Ex. 6.9] with seven allocated names and a spare, run for 200 unfoldings: every unfolding takes exactly 7 parallel steps (between 9 and 11 sequential, averaging the draft’s two erection steps plus eight), and adds exactly 16 names to the table, whatever the depth of the address.

Obligation 14.5 (Phase two’s primitive). Decide whether phase two erects by a single-input context constructor, or by some other construction that meets (D*). Until one is fixed, the compiled images whose width Obligation 13.7 asks to measure do not exist in a correct form. GATE

Obligation 14.6 (Separate erection from computation). When the width of compiled programs is measured, report the steps spent in erection separately from the rest. Under `inst` erection is two steps per instance and perfectly parallel across instances, so it should inflate width rather than depth; the measurement should confirm it. ROUTE

15 Conclusion

The proposal that opened this line of work — encode the reduction graph as an upper triangular matrix, factored into a representation of the rules and repeated application to a vector — survives with one correction and one redirection.

The correction is that triangularity is available exactly on the destructor-free fragment, where the weighted atom-count grading supplies it, and that in general one either moves to the history unfolding or does without. The redirection is that the matrix worth building is not the one indexed by states. It is the incidence of atoms against names, which is the size of the term; on it, redex finding and conflict detection are two sparse Boolean products, and a machine step is a maximal independent set. The factorisation asked for does exist, in the form of one ladder-operator monomial per generated rule, and it is the right way to *think* about the semantics; but the thing to *multiply* is smaller and nearer to hand.

What makes this work at all is the combinators themselves. Because they are name-free and flat, a state is a multiset, structural congruence is free, pattern matching moves into the compiler, and every atom becomes a fixed-width tuple of integers. Those four properties are the ones an accelerator asks for, and the calculus has them by construction rather than by engineering. The remaining risk is not in the representation. It is whether compiled rho programs are wide, and that is a measurement, not a theorem.

Acknowledgement. This note was drafted in dialogue with Claude (Anthropic), which also wrote and ran the reference implementation.

A The reference implementation

Three files accompany the note. `rhocombmat.py` is the machine: an intern table, a state as a multiset of occurrences, incidence matrices, the generated join expressions, the conflict product, a greedy maximal independent set, and the firing rules including the encode and decode paths. `phase2.py` runs the experiments of §14. `experiments.py` builds the four term families, validates the matrix join against naive enumeration, and produces the table of Finding 10.2. The core of the step is short enough to quote:

```

def run_parallel(state, nt, seed=0, cap=10000):
    """Bulk-synchronous: each step fires a maximal independent set."""
    rng = random.Random(seed)
    steps, fired, widths, counts = 0, 0, [], []
    while steps < cap:
        R = redexes_matrix(state, nt) # one join per generated rule
        if not R:
            break
        C, nnz = conflict_graph(R, state) # B B^T
        sel = mis(R, C, rng) # maximal independent set
        widths.append(len(sel))
        counts.append(len(R))
        for r in sel:
            fire(state, nt, R[r]) # remove L, append R
        fired += len(sel)
        steps += 1
    return steps, fired, widths, counts

```

References

- [1] L. G. Meredith and M. Stay. *Name-Free Combinators for the Rho Calculus: a repair, a linear encoding, and the one operation that was missing*. F1R3FLY.io research note, draft 3, September 2026. F1R3FLY-io/publications, directory rho-combinators.
- [2] L. G. Meredith. *A spatial-behavioural logic for the rho combinators*. F1R3FLY.io research note, September 2026. F1R3FLY-io/publications, directory rho-combinators, file rhocomb-logic.tex.
- [3] N. Yoshida. Minimality and separation results on asynchronous mobile processes. *Theoretical Computer Science*, 2002.
- [4] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 2005.
- [5] S. Lybech. Encodability and separation for a reflective higher-order calculus. *EXPRESS/-SOS*, 2022.
- [6] G. Berry and G. Boudol. The chemical abstract machine. *Theoretical Computer Science*, 1992.
- [7] J.-P. Banâtre and D. Le Métayer. Programming by multiset transformation. *Communications of the ACM*, 1993.
- [8] W. Reisig. *Understanding Petri Nets*. Springer, 2013.
- [9] M. Doi. Second quantization representation for classical many-particle systems. *Journal of Physics A*, 1976.
- [10] L. Peliti. Path integral approach to birth-death processes on a lattice. *Journal de Physique*, 1985.
- [11] J. Baez and J. Biamonte. *Quantum Techniques for Stochastic Mechanics*. World Scientific, 2018.
- [12] J. Kepner and J. Gilbert (eds.). *Graph Algorithms in the Language of Linear Algebra*. SIAM, 2011.

- [13] M. Luby. A simple parallel algorithm for the maximal independent set problem. *SIAM Journal on Computing*, 1986.
- [14] H. Q. Ngo, E. Porat, C. Ré and A. Rudra. Worst-case optimal join algorithms. *Journal of the ACM*, 2018.
- [15] L. G. Meredith. *History as an Endofunctor*. F1R3FLY.io working note, July 2026. F1R3FLY.io/publications, directory `history-functor`.
- [16] L. G. Meredith. *Knotted Topoi*. F1R3FLY.io/publications, directory `knotted-topoi`.
- [17] L. G. Meredith. *Graded Where-Clauses*. F1R3FLY.io/publications, directory `fuzzyware`.
- [18] L. G. Meredith. *Stochastic Simulation for Decorated ciGSLTs*, and the `rho-weighted` crate. F1R3FLY.io/publications, directory `rho-life/Gillespie`.